

Lab 2 (Obligatory Assignment 1)

Methods in Artificial Intelligence (INFO180)

Submission Deadline (at Mitt UiB): 11.09.2026 at 14:00

[Note 1]: This is a mandatory assignment and must be completed and approved in order to take exam.

[Note 2]: The updated rules regarding the permitted use of AI for the mandatory assignments in INFO180 are provided in the following folder: *Mitt.UiB -> INFO180 -> Files -> Use of AI in INFO180*

In this assignment, you will program a simple Artificial Intelligence application and test out search algorithms. The program finds solutions for a search in a grid of obstacles. One can imagine a robot that has to move in a grid of size $N \times N$, where many of the fields in the grid are not accessible. In addition, there is a cost to moving into a field.

Here is an 8 x 8 example in the following figure:

START	4	2	3	1	1		2	
	3		4		2		5	5
		5	3		3			3
	4		1		5		3	
	1		5	4	3	1	5	2
	2				5			3
	2		5	2	1			3
	5	4	3	4	4	3	4	1
								GOAL

The goal is to find a path through the grid from the starting position at the top left (coordinates $x = 0$, $y = 0$) and down to the target position at the bottom right (coordinates $x = N-1$, $y = N-1$). In each field in the grid, the robot can move in all directions (right, left, up, down, diagonally), i.e., 8 possibilities.

In the file **Lab2_Files.zip**, available to download at the Lab2 folder at *Mitt.uib*, you can find some files that can be used for running different search algorithms and comparing them on the grid. The programs in these files are not all complete, and some parts might be missing. Hence, your task includes completing those parts. Then, the goal will be to achieve a comparison of Breadth-first search (BFS), Depth-first search (DFS), Lowest-cost-first, and variants of searches with different heuristics. Here is the list of the files:

- **grid_walk_space.py** contains a class that formulates the problem in a search space and finds a solution with the `solve()` method. There are two arguments to set (optional):
 - *frontier*: which is an object of the type of data structure that realizes Breadth-first search (in Python *Queue()*), Depth-first search (in Python *LifoQueue()*, i.e., a stack), and lowest-cost-first search (in Python *PriorityQueue()*).
 - *Heuristic*: which is an object of type *Heuristic*. *Heuristic* contains only one static method that is used together with a priority queue to implement heuristic search.
- **grid_walk_node.py** contains data about a node in the search and functions that make it possible to create neighboring nodes in the grid.
- **grid_walk_path.py** contains data about a path in the search, i.e., nodes in the path, cost for the path, and heuristic value for the path.
- **make_grid.py** contains data and functions that create a random grid that is used in the search. In the function *make_random_grid()*, the size of the grid is set, the probability that a field is unavailable, and a so-called seed for initializing the random number generator. You can vary this in the program line where it says `THE_GRID = make_random_grid()` by giving values to the three optional parameters `size`, `prob` and `seed`.
- **heuristic.py** defines a class *Heuristic*. It is very simple and only has one function that defines a heuristic for the search.
- **zero_h.py** contains a subclass of *Heuristic* that defines a heuristic that is 0 for all nodes. It thus behaves identically to *Heuristic()*.

Now, please complete all the following tasks:

- **Task 1:** In `grid_walk_node.py`, 5 methods for moving in the grid are incomplete and not finished. These methods have a `"return None"` line. You need to complete them all.
- **Task 2:** In `grid_walk_space.py` there is a `"main"` method at the end of the file, which runs a Breadth-first search and prints time spent in seconds, nodes visited, length of path, and cost of path. It should work if you have managed to solve Task 1 satisfactorily, but it can take some time. Complete this program by adding lines that also run the search with Depth-first search and Lowest-cost-first search. Try to compare the search methods based on time, path length, and path cost.
- **Task 3:** Create a new Python file called `max_dim_h.py` that makes a new heuristic class `MaxDimH` (as a subclass of *Heuristic*, similar to `ZeroH` in `zero_h.py`). This heuristic is to be used in a heuristic search. The heuristic must calculate the difference between the x-coordinates and y-coordinates of each position (node) and the target position, and then use the larger of these two values (max value). Test the heuristic as in Task 2 with a program line such as:

```
sp = GridWalkSpace(PriorityQueue(), YourHeuristic())
```

- **Task 4:** Now write a new Python file called `pythagoras_h.py` that creates a new heuristic that calculates what we call the Euclidean distance between each position and the target position. This means that you must use Pythagoras. Then, test this heuristic.
- **Task 5:** Finally, write a new Python file called `approx_max_dim_h.py` that creates another heuristic that does the same as the one in Task 3, but multiplies by 3. Test this heuristic.
- **Task 6:** Try to compare the results for different algorithms. Which of the heuristics you have programmed do you think works best?

Short written questions related to Lectures 1–3

In addition to the programming tasks above, answer the following three questions. Each answer should be approximately 250–400 words. The purpose is to demonstrate that you have read and understood the main concepts from each of the three lectures. Please explain the ideas in your own words and use examples where helpful.

- **Task 7: Lecture 1 / Chapter 1 – Artificial Intelligence and Agents.** Explain what it means to view an AI system as an intelligent agent situated in an environment. In your answer, describe the relationship between the agent and its environment, the role of inputs/perception and actions, and why the design of an agent depends on the task and the environment in which it operates. Give at least one concrete example of an AI agent to illustrate your explanation.
- **Task 8: Lecture 2 / Chapter 3 – Searching for Solutions.** Explain how a problem can be represented as a state-space or graph-search problem and discuss how the choice of search strategy affects the solution process. Compare at least two uninformed search strategies (for example BFS, DFS or Lowest-cost-first search) and explain how heuristic information is used in informed search, particularly in A*. Your answer should also distinguish between path length and path cost and comment on when a search can be considered optimal.
- **Task 9: Lecture 3 / Chapter 4 – Reasoning with Constraints.** Explain the main components of a constraint satisfaction problem (CSP): variables, domains, assignments and constraints. Then describe how consistency algorithms, especially arc consistency or Generalized Arc Consistency (GAC), can reduce the amount of search required by removing values that cannot participate in a solution. Finally, explain why local consistency alone does not always guarantee that a global solution exists.

Suggested length: approximately 250–400 words per question.